

Smart Parking Availability System

IoT-Based Final Project — Spring 2026

Garvin Patel | patelg5@mymail.nku.edu

Northern Kentucky University

The Problem

"Students waste time every day searching for parking — with zero visibility into what's available before entering."

- 🚗 Drivers circle multiple garage levels with no guidance
- ❌ No system to check spot availability before entering
- ⌚ Results in **wasted time**, congestion, and frustration
- 📖 Students arriving **late to class** daily because of parking

This is a real, solvable problem — using IoT.

The Solution

A fully working IoT system that **physically detects cars** and displays live availability on any device.

HC-SR04 Ultrasonic Sensor

↓ detects distance every 3 seconds

XIAO ESP32C6 (WiFi 6)

↓ HTTP POST with JSON payload

Python Flask REST API

↓ validates + stores data

MongoDB Atlas (Cloud Database)

↓ React fetches every 3 seconds

Live React.js Dashboard

↓

Any device anywhere on same network

System Architecture

Layer	Technology	Purpose
Detection	HC-SR04 + ESP32C6	Measure distance, classify spot
Protection	Logic Level Converter	5V → 3.3V signal safety
Transmission	WiFi + HTTP POST	Send JSON data wirelessly
Backend	Python Flask	REST API — receives + processes
Database	MongoDB Atlas	Cloud storage + history
Frontend	React.js	Live animated dashboard

Hardware Setup

COMPONENTS USED:

- **HC-SR04 Ultrasonic Sensor** — fires sound pulse, measures distance to detect cars
- **XIAO ESP32C6** — microcontroller with built-in WiFi 6 + Bluetooth 5.3
- **Logic Level Converter** — critical: converts 5V ECHO signal to 3.3V for ESP32
- **Breadboard + Jumper Wires** — prototyping without soldering

WHY LEVEL CONVERTER WAS NON-NEGOTIABLE:

HC-SR04 ECHO pin → outputs 5V signal (OUTPUT from sensor)

XIAO ESP32C6 GPIO → accepts maximum 3.3V (INPUT to controller)

Direct connection = permanent hardware damage ⚠

How Detection Works

Step 1 → ESP32 sends 10μs HIGH pulse on TRIG pin
Step 2 → HC-SR04 fires ultrasonic sound wave
Step 3 → Sound bounces off object, returns to sensor
Step 4 → ECHO pin stays HIGH for travel duration
Step 5 → $\text{distance} = \text{duration} \times 0.034 \div 2$ (cm)

Result	Threshold	Classification
distance < 50 cm	Car present	 OCCUPIED
distance > 50 cm	Spot empty	 AVAILABLE

Sprint 1 — Hardware + Local Detection

COMPLETED MILESTONES:

- Wired HC-SR04 to Arduino Nano — first sensor test
- Installed CH340 USB driver on Mac for Arduino communication
- Confirmed live distance readings in Serial Monitor
- Identified critical voltage issue — 5V ECHO would damage ESP32
- Added logic level converter to safely protect ESP32C6
- Rewired HC-SR04 directly to XIAO ESP32C6
- Connected to WiFi hotspot successfully
- Built live web dashboard hosted directly from ESP32
- Dashboard shows **OCCUPIED / AVAILABLE** updating every 3 seconds

Sprint 2 — Backend + Full Stack

COMPLETED MILESTONES:

- Python Flask REST API receiving sensor data via HTTP POST
- MongoDB Atlas cloud database storing readings + timestamps
- Full pipeline confirmed: ESP32 → Flask → MongoDB working
- React.js dashboard with **120 animated parking spots**
- SVG cars drawn inside occupied spots
- Live stats: total, available, occupied, occupancy %
- Filter buttons: All Spots / Available Only / Occupied Only
- Hover tooltips showing spot ID, status, distance in cm
- Auto-refreshes from MongoDB via Flask every 3 seconds
- Sensor info panel showing controller, sensor, threshold, update rate

Issues That Could Not Be Fully Solved

Issue	What Happened	Workaround
Flask response -1	ESP32 and laptop on different WiFi subnets — same router, different SSID	Manually matched both to exact same network each session
Mac WiFi not connecting	macOS showed "connected" but had no valid IP — <code>networksetup</code> showed no association	Used USB tethering IP <code>192.0.0.2</code> as temporary fix
Garvin hotspot blocks Flask	ESP32 connected to hotspot but could not reach Flask on laptop even though phone could	Switched to home WiFi instead of hotspot
IP changes every session	Every WiFi change required updating Flask URL in ESP32 code and React manually	No permanent fix — re-uploaded code each time

Learning With AI — Topic 1

Embedded C++ Sensor Programming

What I genuinely learned and applied:

- `pulseIn()` — measures microsecond ECHO duration to calculate distance
- `pinMode()` — every GPIO must be declared INPUT or OUTPUT
- `digitalWrite()` — sends precise HIGH/LOW pulses on TRIG pin
- Distance formula: $\text{duration} \times 0.034 \div 2$
- ECHO is an OUTPUT pin — it sends voltage TO the ESP32 (5V risk)
- Serial Monitor at 115200 baud — only way to debug without a screen
- ATmega328P Old Bootloader — hardware version affects upload method

Learning With AI — Topic 2

IoT Networking — ESP32 to Cloud Pipeline

What I genuinely learned and applied:

- `WiFi.begin()` — ESP32 gets assigned IP on same router subnet
- IP subnets — devices on different ranges **cannot** communicate
- `HttpClient` — sends POST requests with JSON payload every 3s
- Flask `@app.route` — defines REST API endpoints
- `pymongo` + `upsert=True` — update without creating duplicates
- React `useEffect` + `setInterval` — polls Flask API live every 3s
- CORS — allows React (port 3000) to fetch Flask (port 5001)
- HTTP vs HTTPS — ESP32 only supports unencrypted HTTP

Weekly Progress Summary

Week	Milestone
Week 1–2	Project ideation, problem definition, IoT research
Week 3–4	PPP presentation — architecture and hardware plan
Week 5–6	Hardware setup, breadboard wiring, first sensor test
Week 7–8	CH340 driver fix, ESP32C6 integration, voltage protection
Week 9	Sprint 1 complete — WiFi dashboard live, S1P submitted
Week 10–11	Flask API built, MongoDB Atlas connected and working
Week 12–13	React dashboard — 120 spots, SVG cars, live polling
Week 14	Full pipeline tested, code pushed to GitHub
Week 15	Final presentation, rubric submitted, peer evaluations

Live Demo

Watch the system working live:

 <https://youtube.com/shorts/oITPgWEbLkA>

What the demo shows:

1. HC-SR04 sensor detecting distance in real time
2. ESP32C6 sending data to Flask API every 3 seconds
3. MongoDB Atlas storing every reading in the cloud
4. React dashboard — 120 spots updating live
5. Put hand in front → **OCCUPIED** in 3 seconds
6. Remove hand → **AVAILABLE** in 3 seconds

Thank You

Garvin Patel

patelg5@mymail.nku.edu

Northern Kentucky University — Spring 2026

"From zero electronics knowledge to a full-stack IoT system — built with real hardware, real data, and real AI assistance."

GitHub: [your repo link here]

Demo Video: <https://youtube.com/shorts/oITPgWEbLkA>